

A circular wreath of various botanical illustrations, including green ferns, red and orange flowers, and green leaves, surrounding a central white circle.

BCSE301L

Concurrency Control



Agenda

Lock Based Protocols

Timestamp-Based Protocols

Graph Based Protocol

Multiple Granularity

Deadlock Handling

Lock-Based Protocols

- A lock is a mechanism to control concurrent access to a data item
- Data items can be locked in two modes :
 1. **exclusive** (X) mode. Data item can be both read as well as written. X-lock is requested using **lock-X** instruction.
 2. **shared** (S) mode. Data item can only be read. S-lock is requested using **lock-S** instruction.
- Lock requests are made to concurrency-control manager.
- Transaction can proceed only after request is granted.

Lock-Based Protocols

- Lock-compatibility matrix

	S	X
S	true	false
X	false	false

- A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions
- Any number of transactions can hold shared locks on an item,
- But if any transaction holds an exclusive on the item no other transaction may hold any lock on the item.

Lock-Based Protocols

- Example of a transaction performing locking:

```
 $T_2$ : lock-S(A);  
      read (A);  
      unlock(A);  
      lock-S(B);  
      read (B);  
      unlock(B);  
      display(A+B)
```

- Locking as above is not sufficient to guarantee serializability
- Locking as above is not sufficient to guarantee serializability — if A and B get updated in-between the read of A and B , the displayed sum would be wrong.

Lock-Based Protocols

- A **locking protocol** is a **set of rules** followed by all transactions while requesting and releasing locks.
- Locking protocols enforce serializability by restricting the set of possible schedules.

T_1	T_2	concurrency-control manager
lock-X(B)		grant-X(B, T_1)
read(B)		
$B := B - 50$		
write(B)		
unlock(B)		
	lock-S(A)	
	read(A)	grant-S(A, T_2)
	unlock(A)	
	lock-S(B)	
		grant-S(B, T_2)
	read(B)	
	unlock(B)	
	display($A + B$)	
lock-X(A)		grant-X(A, T_1)
read(A)		
$A := A + 50$		
write(A)		
unlock(A)		

Pitfalls of Lock-Based Protocols

- Consider the partial schedule

T_3	T_4
lock-X(B)	
read(B)	
$B := B - 50$	
write(B)	
	lock-S(A)
	read(A)
	lock-S(B)
lock-X(A)	

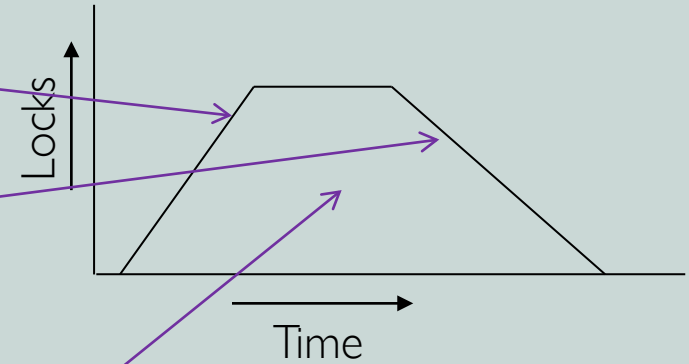
- Neither T_3 nor T_4 can make progress — executing **lock-S(B)** causes T_4 to wait for T_3 to release its lock on B , while executing **lock-X(A)** causes T_3 to wait for T_4 to release its lock on A .
- Such a situation is called a deadlock
 - To handle a deadlock one of T_3 or T_4 must be rolled back and its locks released.

Pitfalls of Lock-Based Protocols

- The potential for **deadlock exists in most locking protocols**. Deadlocks are a necessary evil.
- **Starvation** is also possible if concurrency control manager is badly designed. For example:
 - A transaction may be waiting for an X-lock on an item, **while a sequence of other transactions request and are granted an S-lock on the same item**.
 - The same transaction is repeatedly rolled back due to deadlocks.
- Concurrency control manager can be designed to prevent starvation.

Two-Phase Locking Protocol

- A protocol which ensures conflict-serializable schedules.
- Phase 1: **Growing Phase**
 - Transaction may obtain locks
 - Transaction may not release locks
- Phase 2: **Shrinking Phase**
 - Transaction may release locks
 - Transaction may not obtain locks
- The protocol assures serializability. It can be proved that the transactions can be serialized in the order of their **lock points** (i.e., the point where a transaction acquired its final lock).



Two-Phase Locking Protocol

- Two-phase locking *does not* ensure freedom from deadlocks
- Extensions to basic two-phase locking *needed* to ensure recoverability of freedom from cascading roll-back
 - **Strict two-phase locking:** a transaction must hold all its exclusive locks till it commits / aborts.
 - Ensures recoverability and avoids cascading roll-backs
 - **Rigorous two-phase locking:** a transaction must hold *a//*locks till commit/abort.
 - Transactions can be serialized in the order in which they commit.
- Most databases implement rigorous two-phase locking, *but refer to it as simply two-phase locking*

Two-Phase Locking Protocol

- Two-phase locking is **not a necessary condition for serializability**
 - There are conflict serializable schedules that cannot be obtained if the two-phase locking protocol is used.
- In the absence of extra information (e.g., ordering of access to data), two-phase locking is necessary for conflict serializability *in the following sense*:
 - Given a transaction T_i *that does not follow two-phase locking*, we can find a transaction T_j *that uses two-phase locking*, and a schedule for T_i and T_j that is not conflict serializable.

T_1	T_2
lock-X(B)	
read(B)	
$B := B - 50$	
write(B)	
unlock(B)	
	lock-S(A)
	read(A)
	unlock(A)
	lock-S(B)
	read(B)
	unlock(B)
	display($A + B$)
lock-X(A)	
read(A)	
$A := A + 50$	
write(A)	
unlock(A)	

Locking Protocol & Lock Conversions

- Given a locking protocol (such as 2PL)
 - A schedule S is **legal** under a locking protocol if it can be generated by a set of transactions that follow the protocol
 - A protocol **ensures** serializability if all legal schedules under that protocol are serializable
- Two-phase locking protocol with lock conversions:
 - Growing Phase:
 - can acquire a lock-S on item
 - can acquire a lock-X on item
 - can **convert** a lock-S to a lock-X (**upgrade**)
 - Shrinking Phase:
 - can release a lock-S
 - can release a lock-X
 - can convert a lock-X to a lock-S (**downgrade**)
- This protocol ensures serializability

Automatic Acquisition of Locks

- A transaction T_i issues the standard read/write instruction, without explicit locking calls.
- The operation **read**(D) is processed as:
 - if T_i has a lock on D then
 - read(D)
 - else begin
 - if necessary wait until no other transaction has a **lock-X** on D
 - grant T_i a **lock-S** on D ;
 - read(D)
 - end

Automatic Acquisition of Locks

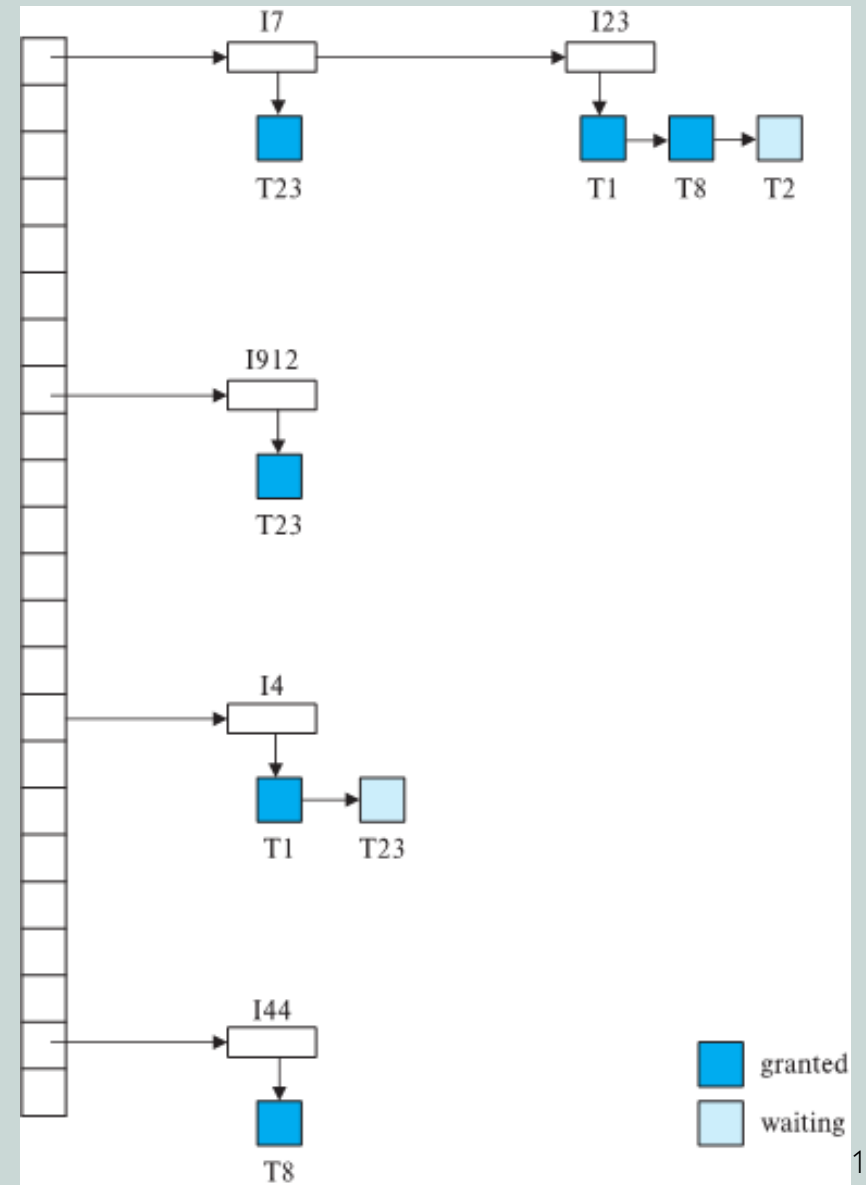
- The operation **write**(D) is processed as:
 - if T_i has a **lock-X** on D then
 - write(D)
 - else begin
 - if necessary wait until no other trans. has any lock on D ,
 - if T_i has a **lock-S** on D
 - then
 - upgrade** lock on D to **lock-X**
 - else
 - grant** T_i a **lock-X** on D
 - write(D)
 - end;
- All locks are released after commit or abort

Implementation of Locking

- A Lock manager can be implemented as a separate process to which transactions send lock and unlock requests
- The lock manager replies to a lock request by sending a lock grant messages (or a message asking the transaction to roll back, in case of a deadlock)
- The requesting transaction waits until its request is answered
- The lock manager maintains a datastructure called a **lock table** to record granted locks and pending requests
- The lock table is usually implemented as an in-memory hash table indexed on the name of the data item being locked

Lock Table

- Dark rectangles indicate granted locks, light colored ones indicate waiting requests
- Lock table also records the type of lock granted or requested
- New request is added to the end of the queue of requests for the data item, and granted if it is compatible with all earlier locks
- Unlock requests result in the request being deleted, and later requests are checked to see if they can now be granted
- If transaction aborts, all waiting or granted requests of the transaction are deleted
 - lock manager may keep a list of locks held by each transaction, to implement this efficiently



Exercise

- Consider the following two transactions
- T1: Read (X)
Read (Y)
if $X = 0$ then $Y = Y + 1$;
Write (Y)
- T2 Read (Y)
Read (X)
if $Y = 0$ then $X = X + 1$;
Write (X)
- Add lock and unlock instruction to T1 and T2, they use two-phase locking protocol. Can the execution of these transactions result in deadlock?

Excerise

• T1:

lock- S(X)

Read (X)

lock- X(Y)

Read (Y)

if $X = 0$ then $Y = Y + 1$;

Write (Y)

unlock (X)

unlock (Y)

• T2

lock- S(Y)

Read (Y)

lock- X(X)

Read (X)

if $Y = 0$ then $X = X + 1$;

Write (X)

unlock (Y)

unlock (X)

Exercise

- T1:

lock- S(X)

Read (X)

lock- X(Y)

- T2

lock- S(Y)

Read (Y)

lock- X(X)

Transactions are now in deadlock



Timestamp Based Concurrency Control



Timestamp-Based Protocols

- Each transaction T_i is issued a **timestamp** $TS(T_i)$ when it enters the system.
 - Each transaction has a *unique* timestamp
 - Newer transactions have timestamps strictly greater than earlier ones
 - Timestamp could be based on a logical counter
 - Real time may not be unique
 - Can use (wall-clock time, logical counter) to ensure
- Timestamp-based protocols manage concurrent execution such that
time-stamp order = serializability order
- Several alternative protocols based on timestamps

Timestamp-Based Protocols

The **timestamp ordering (TSO) protocol**

- Maintains for each data Q two timestamp values:
 - **W-timestamp(Q)** is the largest time-stamp of any transaction that executed **write(Q)** successfully.
 - **R-timestamp(Q)** is the largest time-stamp of any transaction that executed **read(Q)** successfully.
- Imposes rules on read and write operations to ensure that
 - Any conflicting operations are executed in timestamp order
 - Out of order operations cause transaction rollback

Timestamp-Based Protocols

- Suppose a transaction T_i issues a read(Q)
 1. If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i needs to read a value of Q that was already overwritten.
 - Hence, the read operation is rejected, and T_i is rolled back.
 2. If $TS(T_i) \geq W\text{-timestamp}(Q)$, then the **read** operation is executed, and $R\text{-timestamp}(Q)$ is set to $\max(R\text{-timestamp}(Q), TS(T_i))$.

Timestamp-Based Protocols

- Suppose that transaction T_i issues write(Q).
 1. If $TS(T_i) < R\text{-timestamp}(Q)$, then the value of Q that T_i is producing was needed previously, and the system assumed that that value would never be produced.
 - Hence, the **write** operation is rejected, and T_i is rolled back.
 2. If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of Q .
 - Hence, this **write** operation is rejected, and T_i is rolled back.
 3. Otherwise, the **write** operation is executed, and $W\text{-timestamp}(Q)$ is set to $TS(T_i)$.

Example of Timestamp-Based Protocols

- Is this schedule valid under TSO?
- Assume that initially:
 - $R\text{-TS}(A) = W\text{-TS}(A) = 0$
 - $R\text{-TS}(B) = W\text{-TS}(B) = 0$
- Assume $TS(T_{25}) = 25$ and $TS(T_{26}) = 26$

T_{25}	T_{26}
read(B)	read(B) $B := B - 50$ write(B)
read(A)	read(A)
display($A + B$)	$A := A + 50$ write(A) display($A + B$)

- How about this one,
where initially
 $R\text{-TS}(Q) = W\text{-TS}(Q) = 0$

T_{27}	T_{28}
read(Q)	write(Q)
write(Q)	

Example of Timestamp-Based Protocols

- A partial schedule for several data items for transactions with timestamps 1, 2, 3, 4, 5, with all R-TS and W-TS = 0 initially

T_1	T_2	T_3	T_4	T_5
read (Y)	read (Y)	write (Y) write (Z)		read (X)
	read (Z) abort			read (Z)
read (X)		write (W) abort	read (W)	
				write (Y) write (Z)

Correctness of Timestamp-Based Protocols

- The timestamp-ordering protocol guarantees serializability since all the arcs in the precedence graph are of the form:



Thus, there will be no cycles in the precedence graph

- Timestamp protocol ensures freedom from deadlock as no transaction ever waits.
- But the schedule may not be cascade-free, and may not even be recoverable.

Recoverability and Cascade Freedom

- Solution 1:
 - A transaction is structured such that its writes are all performed at the end of its processing
 - All writes of a transaction form an atomic action; no transaction may execute while a transaction is being written
 - A transaction that aborts is restarted with a new timestamp
- Solution 2:
 - Limited form of locking: wait for data to be committed before reading it
- Solution 3:
 - Use commit dependencies to ensure recoverability

Thomas' Write Rule

- Modified version of the timestamp-ordering protocol in which obsolete **write** operations may be ignored under certain circumstances.
- When T_i attempts to write data item Q , if $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of $\{Q\}$.
 - Rather than rolling back T_i as the timestamp ordering protocol would have done, this {write} operation can be ignored.
- Otherwise this protocol is the same as the timestamp ordering protocol.
- Thomas' Write Rule allows greater potential concurrency.
 - Allows some view-serializable schedules that are not conflict-serializable.



Graph Based Concurrency Control



Graph Based Protocol for Concurrency Control

- Graph-based concurrency control protocols are used in database management systems (DBMS) to ensure that concurrent transactions do not interfere with each other and maintain the consistency of the database.
- Transactions are represented as nodes in a graph, and the *conflicts between transactions are represented as edges* between the nodes.
- A conflict between two transactions occurs when they access the same data item, and at least one of the transactions performs a write operation.

Graph Based Protocol for Concurrency Control

- The graph-based concurrency control protocol operates as follows:
- When a transaction starts, a node is created in the graph representing the transaction.
- When a transaction accesses a data item, it acquires a shared or exclusive lock on the item, and the corresponding node is added to the graph.
- If a transaction tries to acquire an exclusive lock on an item that is already locked by another transaction, a conflict edge is added between the nodes representing the two transactions.
- When a transaction completes, all the edges incident on its node are removed from the graph.

Graph Based Protocol for Concurrency Control

- Before granting a lock to a transaction, the protocol checks the graph for any cycles.
- If a cycle exists, it means that there is a conflict between transactions, and one of them needs to be rolled back to break the cycle.

Graph Based Protocol for Concurrency Control

- **Advantage of graph-based concurrency control:**
 - Handle more complex transaction dependencies than other protocols, such as two-phase locking.
- **Disadvantage:**
 - It can also be more computationally expensive and may not scale well for large databases with a large number of transactions.
- Therefore, the choice of concurrency control protocol depends on the specific requirements and characteristics of the database and the application.

Graph Based Protocol for Concurrency Control

- Graph-Based Protocols are used as an alternative to 2-PL.
- Tree Based Protocols is a simple implementation of Graph Based Protocol.
- A prerequisite of this protocol is that we know the order to access a Database Item.
- For this, we implement a **Partial Ordering** on a set of the Database Items (D) $\{d_1, d_2, d_3, \dots, d_n\}$.
- The protocol following the implementation of Partial Ordering is stated as-
- If $d_i \rightarrow d_j$ then any transaction accessing both d_i and d_j must access d_i before accessing d_j .
- Implies that the set D may now be viewed as a directed acyclic graph (DAG), called a database graph.

Tree Based Protocol

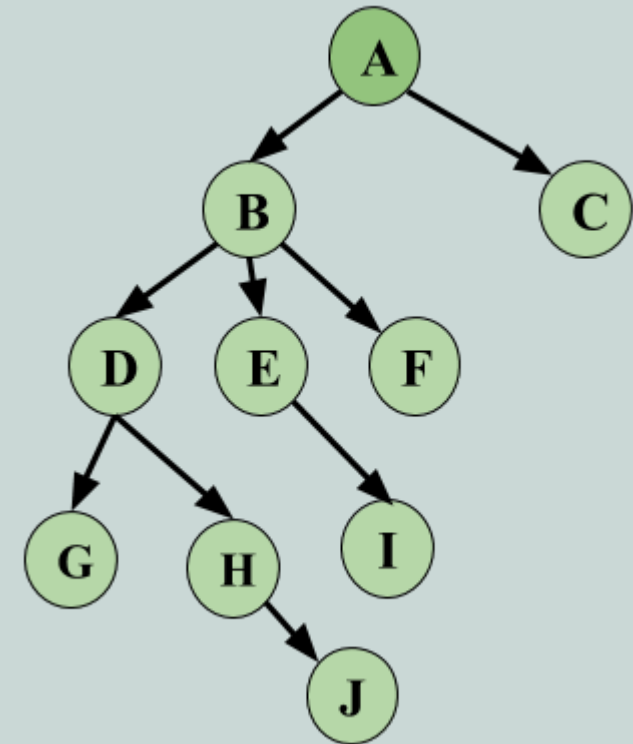
- Partial Order on Database items determines a tree-like structure.
- Only Exclusive Locks are allowed.
- The first lock by T_i may be on any data item. Subsequently, a data Q can be locked by T_i only if the parent of Q is currently locked by T_i .
- Data items can be unlocked at any time.

Tree Based Protocol

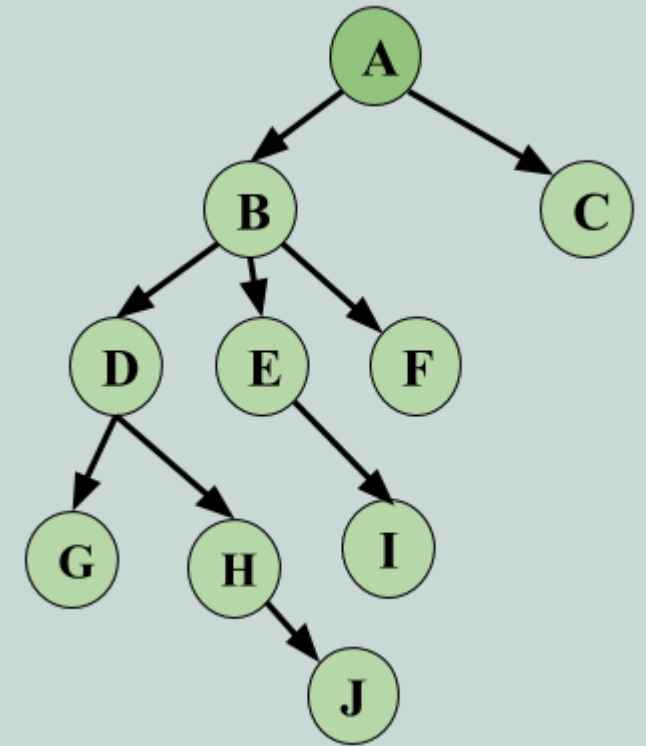
- Tree based Protocol ensures Conflict Serializability and Deadlock Free schedule.
- We need not wait for unlocking a Data item as we did in 2-PL protocol, thus increasing the concurrency.

Example

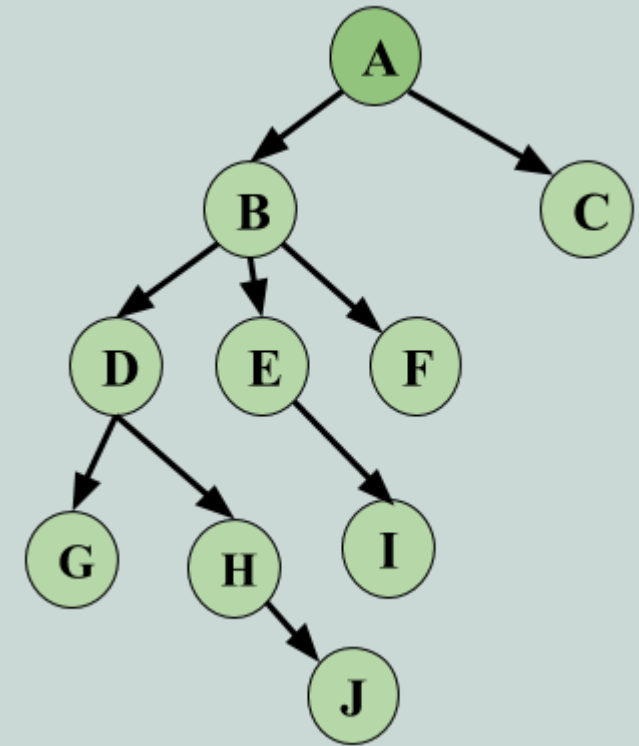
- Database Graph which will be used as a reference for locking the items subsequently is given below.
- There are 3 transactions and given schedule S is (Lock₁ – is lock operation in Transaction T1).
- Lock₁-X(A), Lock₁-X(B), Lock₂-X(D), Lock₂-X(H), Unlock₂-X(D), Lock₁-X(E), Lock₁-X(D), Unlock₁-X(B), Unlock₁-X(E), Lock₃-X(B), Lock₃-X(E), Unlock₂-X(H), Lock₁-X(B), Unlock₁-X(D), Unlock₃-X(E), Unlock₃-X(B), Unlock₁-X(B), Unlock₁-X(A).



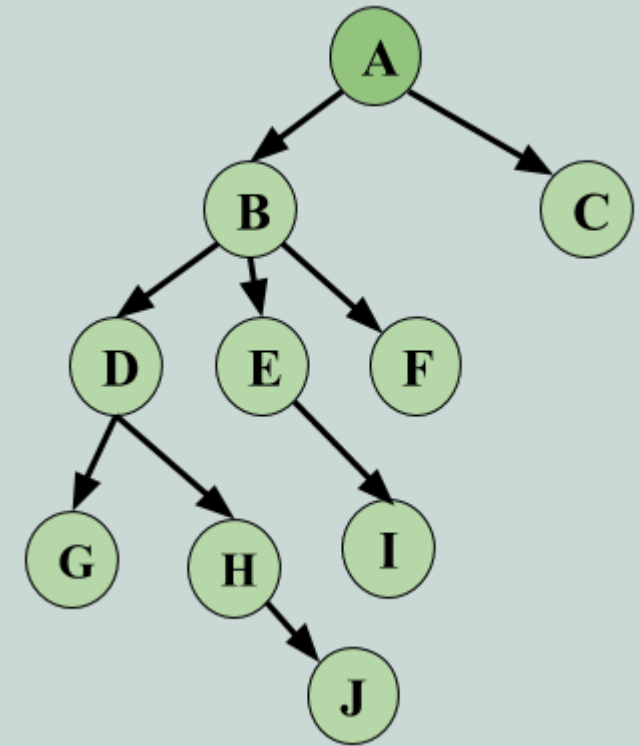
Transaction 1	Transaction 2	Transaction 3	Action
Lock-X(A)			Lock is Granted for T1 on A
Lock-X(B)			A is locked by T1 so Grant Lock B
	Lock-X(D)		Conflict Since B is holding by T1
	Lock-X(H)		T2 is rolledback
	Unlock-X(D)		
Lock-X(E)			B is locked by T1 so Grant Lock E
Lock-X(D)			B is locked by T1 so Grant Lock D
Unlock-X(B)			B is unlocked
Unlock-X(E)			E is inlocked
		Lock-X(B)	Conflict Since B is holding by T1
		Lock-X(E)	T3 is rolledback
	Unlock-X(H)		
Lock-X(B)			
Unlock-X(D)			
		Unlock-X(E)	
		Unlock-X(B)	
Unlock-X(B)			
Unlock-X(A)			



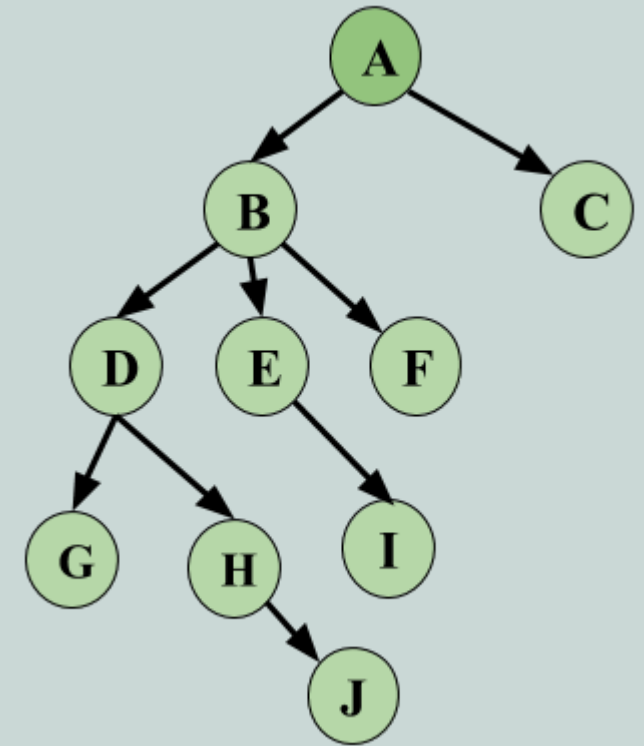
Transaction 1	Transaction 2	Transaction 3	Action
Lock-X(A)			Lock is Granted for T1 on A
Lock-X(B)			A is locked by T1 so Grant Lock B
Lock-X(E)			B is locked by T1 so Grant Lock E
Lock-X(D)			B is locked by T1 so Grant Lock D
Unlock-X(B)			B is unlocked
	Lock-X(D)		Conflict since D is locked by T1
	Lock-X(H)		
	Unlock-X(D)		
Unlock-X(E)			
		Lock-X(B)	
		Lock-X(E)	
	Unlock-X(H)		
Lock-X(B)			
Unlock-X(D)			
		Unlock-X(E)	
		Unlock-X(B)	
Unlock-X(B)			
Unlock-X(A)			



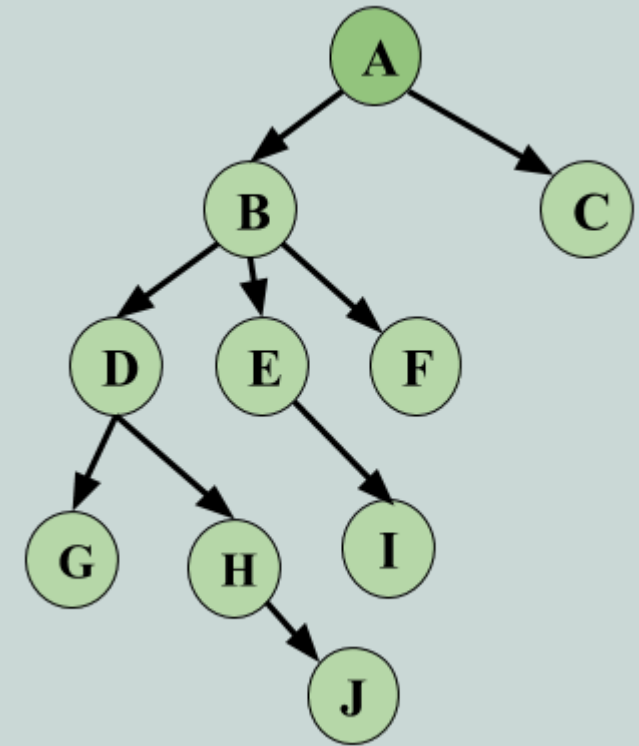
Transaction 1	Transaction 2	Transaction 3	Action
Lock-X(A)			Lock is Granted for T1 on A
Lock-X(B)			A is locked by T1 so Grant Lock B
Lock-X(E)			B is locked by T1 so Grant Lock E
Lock-X(D)			B is locked by T1 so Grant Lock D
Unlock-X(B)			B is unlocked
Unlock-X(E)			E is unlocked
Lock-X(B)			A is locked by T1 so Grant Lock B
Unlock-X(D)			D is unlocked
	Lock-X(D)		Still it results in Rollback of T2 & T3
	Lock-X(H)		
	Unlock-X(D)		
		Lock-X(B)	
		Lock-X(E)	
	Unlock-X(H)		
		Unlock-X(E)	
		Unlock-X(B)	
Unlock-X(B)			
Unlock-X(A)			



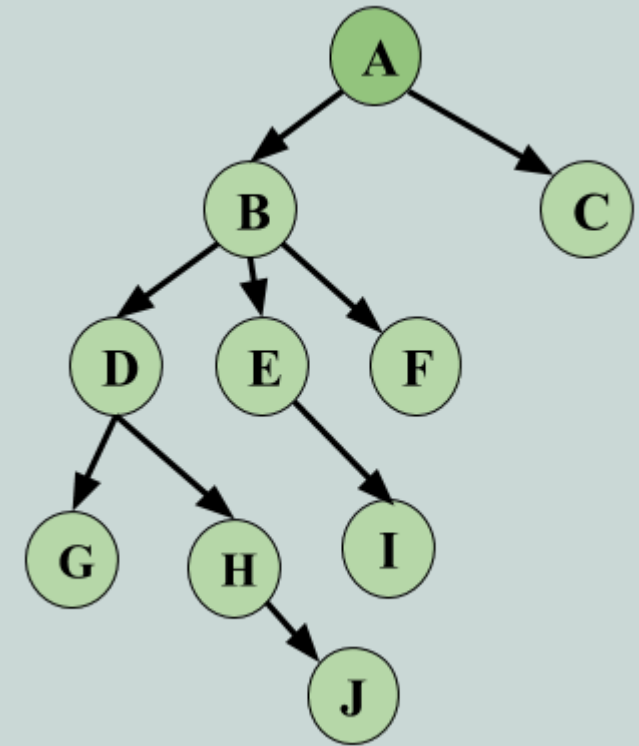
Transaction 1	Transaction 2	Transaction 3	Action
Lock-X(A)			Lock is Granted for T1 on A
Lock-X(B)			A is locked by T1 so Grant Lock B
	Lock-X(D)		Conflict Since B is holding by T1
	Lock-X(H)		T2 is rollbacked
	Unlock-X(D)		
Lock-X(E)			B is locked by T1 so Grant Lock E
Lock-X(D)			B is locked by T1 so Grant Lock D
Unlock-X(B)			B is unlocked
Unlock-X(E)			E is inlocked
		Lock-X(B)	Conflict Since B is holding by T1
		Lock-X(E)	T3 is rollbacked
	Unlock-X(H)		
Lock-X(B)			
Unlock-X(D)			
		Unlock-X(E)	
		Unlock-X(B)	
Unlock-X(B)			
Unlock-X(A)			



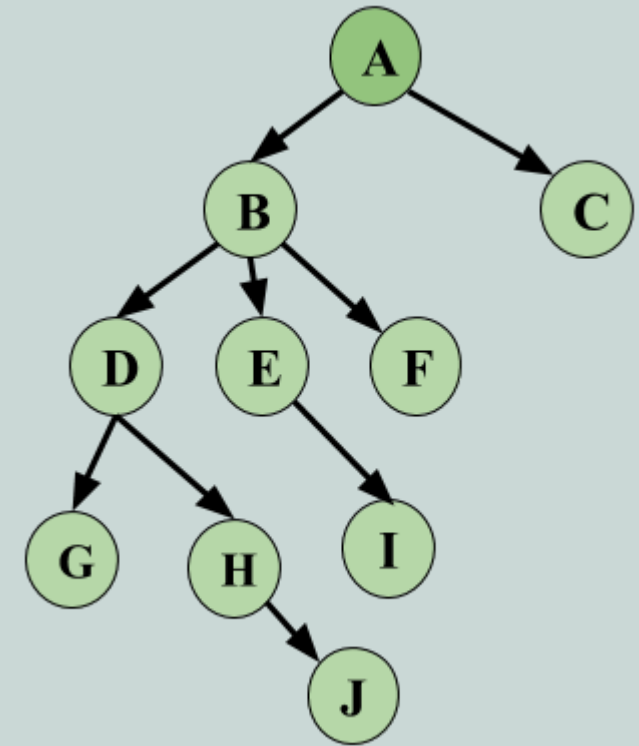
Transaction 1	Transaction 2	Transaction 3	Action
	Lock-X(D)		Lock Granted for T2
	Lock-X(H)		Lock Granted for T2
	Unlock-X(D)		D is unlocked
Lock-X(A)			Lock Granted for T1 on A
Lock-X(B)			A is locked by T1 so Grant Lock B
Lock-X(E)			B is locked by T1 so Grant Lock E
Lock-X(D)			B is locked by T1 so Grant Lock D
Unlock-X(B)			B is unlocked
Unlock-X(E)			E is inlocked
		Lock-X(B)	Conflict Since B is holding by T1
		Lock-X(E)	T3 is rollbacked
	Unlock-X(H)		
Lock-X(B)			
Unlock-X(D)			
		Unlock-X(E)	
		Unlock-X(B)	
Unlock-X(B)			
Unlock-X(A)			



Transaction 1	Transaction 2	Transaction 3	Action
	Lock-X(D)		Lock Granted for T2
	Lock-X(H)		Lock Granted for T2
	Unlock-X(D)		D is unlocked
	Unlock-X(H)		H is unlocked
Lock-X(A)			Lock Granted for T1 on A
Lock-X(B)			A is locked by T1 so Grant Lock B
Lock-X(E)			B is locked by T1 so Grant Lock E
Lock-X(D)			B is locked by T1 so Grant Lock D
Unlock-X(B)			B is unlocked
Unlock-X(E)			E is inlocked
		Lock-X(B)	Conflict Since B is holding by T1
		Lock-X(E)	T3 is rollbacked
Lock-X(B)			
Unlock-X(D)			
		Unlock-X(E)	
		Unlock-X(B)	
Unlock-X(B)			
Unlock-X(A)			



a	Transaction 2	Transaction 3	Action
	Lock-X(D)		Lock Granted for T2
	Lock-X(H)		Lock Granted for T2
	Unlock-X(D)		D is unlocked
	Unlock-X(H)		H is unlocked
Lock-X(A)			Lock Granted for T1 on A
Lock-X(B)			A is locked by T1 so Grant Lock B
Lock-X(E)			B is locked by T1 so Grant Lock E
Lock-X(D)			B is locked by T1 so Grant Lock D
Unlock-X(B)			B is unlocked
Unlock-X(E)			E is inlocked
Lock-X(B)			A is locked by T1 so Grant Lock B
Unlock-X(D)			D is unlocked
Unlock-X(B)			B is unlocked
Unlock-X(A)			A is unlocked
		Lock-X(B)	Lock is Granted for T3
		Lock-X(E)	Lock is Granted for T3
		Unlock-X(E)	E is unlocked
		Unlock-X(B)	B is unlocked



Tree Based Protocol

- So the given schedule is Conflict serializable.
- Serializability for Locks can be written as $T2 \rightarrow T1 \rightarrow T3$.
- Data items Locked and Unlocked are following the same rule as given above and follow the Database Graph.

Tree Based Protocol

- Advantage:
 - Ensures Conflict Serializable Schedule.
 - Ensures Deadlock Free Schedule
 - Unlocking can be done anytime
- Disadvantage:
 - Unnecessary locking overheads may happen sometimes, like if we want both D and E, then at least we have to lock B to follow the protocol.
 - Cascading Rollbacks is still a problem. We don't follow a rule of when Unlock operation may occur so this problem persists for this protocol. Overall this protocol is mostly known and used for its unique way of implementing Deadlock Freedom.



Multiple Granularity



Multiple Granularity

- **Granularity:** It is the size of data item allowed to lock.
- **Multiple Granularity:**
 - It can be defined as hierarchically breaking up the database into blocks which can be locked.
 - The Multiple Granularity protocol enhances concurrency and reduces lock overhead.
 - It maintains the track of what to lock and how to lock.
 - It makes easy to decide either to lock a data item or to unlock a data item. This type of hierarchy can be graphically represented as a tree.

Multiple Granularity

- For example: Consider a tree which has four levels of nodes.
 - The first level or higher level shows the entire database.
 - The second level represents a node of type area. The higher level database consists of exactly these areas.
 - The area consists of children nodes which are known as files. No file can be present in more than one area.
 - Finally, each file contains child nodes known as records. The file has exactly those records that are its child nodes. No records represent in more than one file.
- Hence, the levels of the tree starting from the top level are as follows:
 - Database
 - Area
 - File
 - Record

Multiple Granularity

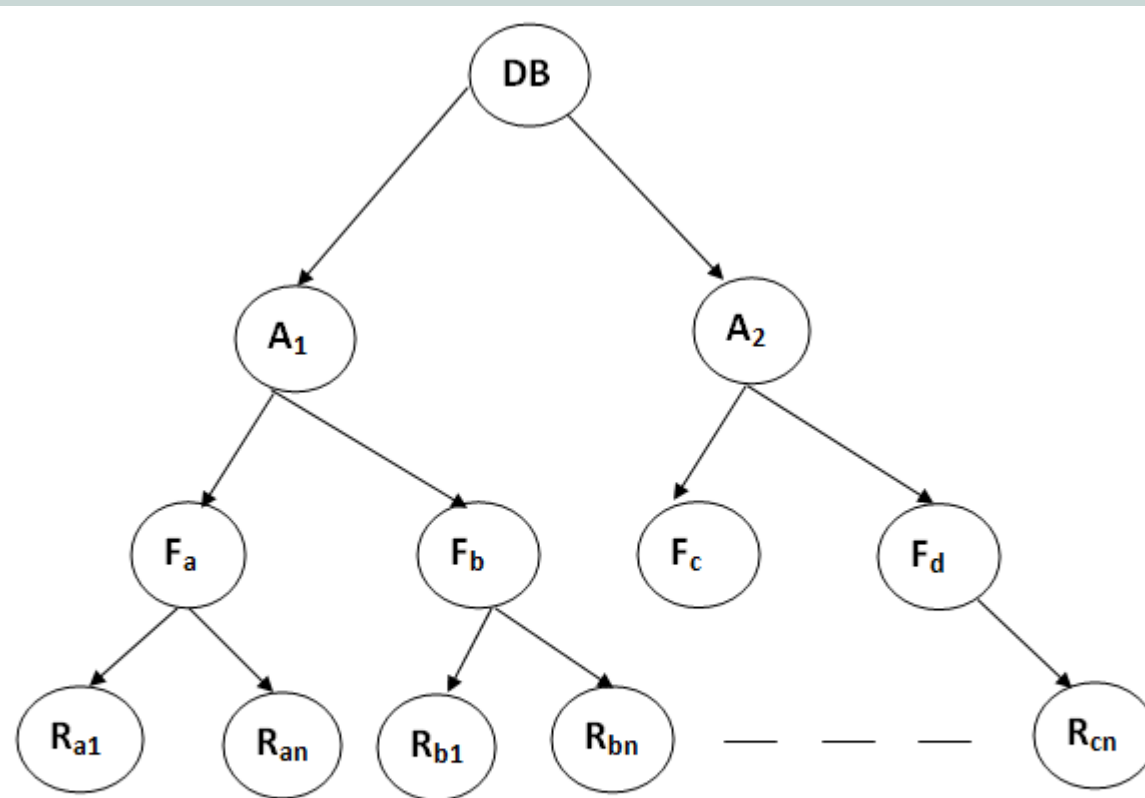


Figure: Multi Granularity tree Hierarchy

Multiple Granularity

- There are three additional lock modes with multiple granularity:
- Intention Mode Lock
- **Intention-shared (IS):** It contains explicit locking at a lower level of the tree but only with shared locks.
- **Intention-Exclusive (IX):** It contains explicit locking at a lower level with exclusive or shared locks.
- **Shared & Intention-Exclusive (SIX):** In this lock, the node is locked in shared mode, and some node is locked in exclusive mode by the same transaction.

Multiple Granularity

- Compatibility Matrix with Intention Lock Modes:

	IS	IX	S	S IX	X
IS	✓	✓	✓	✓	×
IX	✓	✓	×	×	×
S	✓	×	✓	×	×
S IX	✓	×	×	×	×
X	×	×	×	×	×

Multiple Granularity

- It uses the intention lock modes to ensure serializability. It requires that if a transaction attempts to lock a node, then that node must follow these protocols:
 - Transaction T1 should follow the lock-compatibility matrix.
 - Transaction T1 firstly locks the root of the tree. It can lock it in any mode.
 - If T1 currently has the **parent of the node locked in either IX or IS mode**, then the transaction T1 will lock a node in S or IS mode only.
 - If T1 currently has the **parent of the node locked in either IX or SIX modes**, then the transaction T1 will lock a node in X, SIX, or IX mode only.
 - If T1 has not previously unlocked any node only, then the Transaction T1 can lock a node.
 - If T1 currently has none of the children of the node-locked only, then Transaction T1 will unlock a node.

Node to locked	Parent of node to locked
S mode	IS or IX or SIX mode
X mode	IX or SIX mode
IS mode	IS or IX mode
IX mode	IX or SIX mode
SIX mode	IX or SIX mode

Multiple Granularity

- Observe that in multiple-granularity, the locks are acquired in top-down order, and locks must be released in bottom-up order.
 - If transaction T1 reads record R_{a9} in file Fa, then transaction T1 needs to lock the database, area A1 and file Fa in IX mode. Finally, it needs to lock R_{a2} in S mode.
 - If transaction T2 modifies record R_{a9} in file Fa, then it can do so after locking the database, area A1 and file Fa in IX mode. Finally, it needs to lock the R_{a9} in X mode.
 - If transaction T3 reads all the records in file Fa, then transaction T3 needs to lock the database, and area A in IS mode. At last, it needs to lock Fa in S mode.
 - If transaction T4 reads the entire database, then T4 needs to lock the database in S mode.

Example Multiple Granularity

- Consider the database that is organized in terms of the following hierarchy of objects:
- The database itself is an object D and it contains two files F1 and F2 each of which contains 1000 pages (P1...P1000 and P1001..P2000 respectively). Each page contains 100 records, and records are identified as P:R, where P is the page identifier and R is the slot of record on that page.
- Multiple granularity locking is used, with S, X, IS, IX, SIX locks and DB level, file level, page level and record level locking.
- Which of the following locking sequence of locking is correct for following operation?
- Read page P15 through P990?
 - D-IS F1-IS P15-S
 - D:IS F1-IS
 - **D-IS F1-S**
 - D-IS F1-IS P990:S

Example Multiple Granularity

- Consider the database that is organized in terms of the following hierarchy of objects:
- The database itself is an object D and it contains two files F1 and F2 each of which contains 1000 pages (P1...P1000 and P1001..P2000 respectively). Each page contains 100 records, and records are identified as P:R, where P is the page identifier and R is the slot of record on that page.
- Multiple granularity locking is used, with S, X, IS, IX, SIX locks and DB level, file level, page level and record level locking.
- Which of the following locking sequence of locking is correct for following operation?
- Read page P1250:10?
 - D-IS F2-IS P1250-IS P1250:10-S
 - D-IS F1-IS P1250-IS P1250:10-S
 - D-IX F2-IS P1250-IS P1250:10-IS
 - D-IX F2-IX P1250-IX P1250:10-S

Multiple Granularity

- Allow data items to be various sizes and define a hierarchy of data granularities, where the small granularities are nested within larger ones.
- Granularity of locking (level in tree where locking is done)
 - **Fine granularity** (lower in tree)
 - High concurrency, high locking overhead
 - **Coarse granularity** (higher in tree)
 - Low locking overhead, low concurrency

Example Multiple Granularity

- Consider the database that is organized in terms of the following hierarchy of objects:
- The database itself is an object D and it contains two files F1 and F2 each of which contains 1000 pages (P1...P1000 and P1001..P2000 respectively). Each page contains 100 records, and records are identified as P:R, where P is the page identifier and R is the slot of record on that page.
- Multiple granularity locking is used, with S, X, IS, IX, SIX locks and DB level, file level, page level and record level locking.
- Which of the following locking sequence of locking is correct for following operation?
- Delete record P1500:99?
 - D-IS F2-IS P1500-IX P1500:99-X
 - **D-IX F2-IX P1500-IX P1500:99-X**
 - D-IS F2-IS P1500-IS P1500:99-X
 - D-IS F2-IX P1500-IX P1500:99-X

Example Multiple Granularity

- Consider the database that is organized in terms of the following hierarchy of objects:
- The database itself is an object D and it contains two files F1 and F2 each of which contains 1000 pages (P1...P1000 and P1001..P2000 respectively). Each page contains 100 records, and records are identified as P:R, where P is the page identifier and R is the slot of record on that page.
- Multiple granularity locking is used, with S, X, IS, IX, SIX locks and DB level, file level, page level and record level locking.
- Explain how locks will be granted for each transaction and which transaction will be rolled back?
- T1 wants to read record P1500:99
- T2 wants to modify record P1500:99
- T3 wants to delete record P1500:100
- T4 wants to read all records of P1500
- T5 wants to read the entire DB

Solution: only T1 and T3 alone will get executed, where T2, T4, T5 will be rolled back.



Multiple Granularity



Deadlock Handling

- System is deadlocked if there is a set of transactions such that every transaction in the set is waiting for another transaction in the set.

T_3	T_4
lock-X(B) read(B) $B := B - 50$ write(B)	
	lock-S(A) read(A) lock-S(B)
lock-X(A)	

Deadlock Handling

- Consider the following two transactions:

T_1 : write (X)
 write(Y)

T_2 : write(Y)
 write(X)

- Schedule with deadlock

T_1	T_2
lock-X on X write (X) wait for lock-X on Y	lock-X on Y write (X) wait for lock-X on X

Deadlock Handling

- *Deadlock prevention* protocols ensure that the system will never enter into a deadlock state. Some prevention strategies:
 - Require that each transaction locks all its data items before it begins execution (pre-declaration).
 - Impose partial ordering of all data items and require that a transaction can lock data items only in the order specified by the partial order.

Deadlock Prevention Strategies

- **wait-die** scheme — **non-preemptive**
 - Older transaction may wait for younger one to release data item.
 - Younger transactions never wait for older ones; they **are rolled back instead**.
 - A transaction may die several times before acquiring a lock
- **wound-wait** scheme — **preemptive**
 - Older transaction *wounds* (forces rollback) of younger transaction instead of waiting for it.
 - Younger transactions may wait for older ones.
 - Fewer rollbacks than *wait-die* scheme.
- In both schemes, ***a rolled back transactions is restarted*** with its original timestamp.
 - Ensures that older transactions have precedence over newer ones, and starvation is thus avoided.

Deadlock Prevention Strategies

- Timeout-Based Schemes:

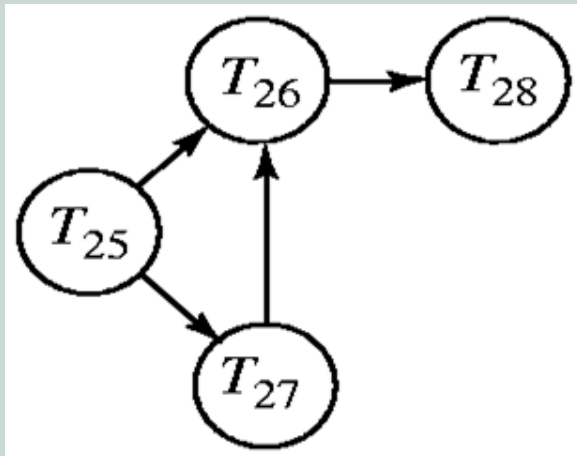
- A transaction waits for a lock only for a specified amount of time. After that, the wait times out and the transaction is rolled back.
- Ensures that deadlocks get resolved by timeout if they occur
- Simple to implement
- But may roll back transaction unnecessarily in absence of deadlock
 - Difficult to determine good value of the timeout interval.
- Starvation is also possible

Deadlock Detection

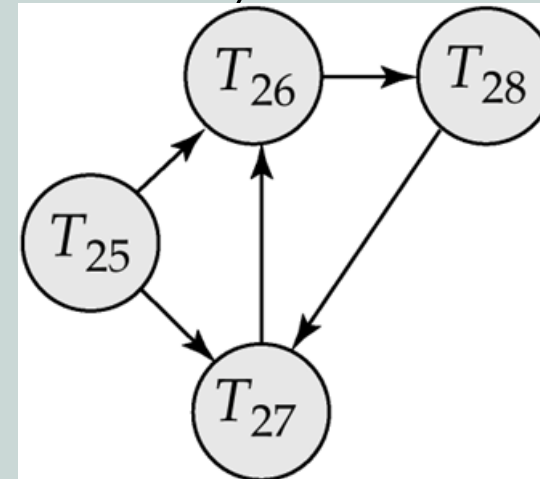
- Deadlocks can be described as a **wait-for graph**, which consists of a pair $G = (V, E)$,
- **Wait-for graph**
 - Vertices: transactions
 - Edge from $T_i \rightarrow T_j$: if T_i is waiting for a lock held in conflicting mode by T_j
- The system is in a **deadlock state if and only if the wait-for graph has a cycle.**
- Invoke a deadlock-detection algorithm periodically to look for cycles.

Deadlock Detection

- If $T_i \rightarrow T_j$ is in E , then there is a directed edge from T_i to T_j , implying that T_i is waiting for T_j to release a data item.
- When T_i requests a data item currently being held by T_j , then the edge $T_i \rightarrow T_j$ is inserted in the wait-for graph. This edge is removed only when T_j is no longer holding a data item needed by T_i .
- The system is in a deadlock state if and only if the wait-for graph has a cycle. Must invoke a deadlock-detection algorithm periodically to look for cycles.



Wait-for graph without a cycle



Wait-for graph with a cycle

Deadlock Recovery

- When deadlock is detected :
 - Some **transaction will have to rolled back** (made a **victim**) to break deadlock cycle.
 - Select that transaction as victim that will incur minimum cost
 - Rollback -- **determine how far to roll back transaction**
 - **Total rollback**: Abort the transaction and then restart it.
 - **Partial rollback**: Roll back victim transaction only as far as necessary to release locks that another transaction in cycle is waiting for
- Starvation can happen
 - One solution: oldest transaction in the deadlock set is never chosen as victim



Multiversion Schemes



Multiversion Schemes

- Multiversion schemes keep old versions of data item to increase concurrency.
 - Multiversion Timestamp Ordering
 - Multiversion Two-Phase Locking
- Each successful **write** results in the creation of a new version of the data item written.
- Use timestamps to label versions.
- When a **read**(Q) operation is issued, select an appropriate version of Q based on the timestamp of the transaction, and return the value of the selected version.
- **reads** never have to wait as an appropriate version is returned immediately.

Multiversion Timestamp Ordering

- Each data item Q has a sequence of versions $\langle Q_1, Q_2, \dots, Q_m \rangle$. Each version Q_k contains three data fields:
 - **Content** -- the value of version Q_k .
 - **W-timestamp**(Q_k) -- timestamp of the transaction that created (wrote) version Q_k
 - **R-timestamp**(Q_k) -- largest timestamp of a transaction that successfully read version Q_k
- when a transaction T_i creates a new version Q_k of Q , Q_k 's W-timestamp and R-timestamp are initialized to $TS(T_i)$.
- R-timestamp of Q_k is updated whenever a transaction T_j reads Q_k , and $TS(T_j) > \text{R-timestamp}(Q_k)$.

Multiversion Timestamp Ordering

- The multiversion timestamp scheme presented next ensures serializability.
- Suppose that transaction T_i issues a **read**(Q) or **write**(Q) operation. Let Q_k denote the version of Q whose write timestamp is the largest write timestamp less than or equal to $TS(T_i)$.
 1. If transaction T_i issues a **read**(Q), then the value returned is the content of version Q_k .
 2. If transaction T_i issues a **write**(Q), and if $TS(T_i) < R\text{-timestamp}(Q_k)$, then transaction T_i is rolled back. Otherwise, if $TS(T_i) = W\text{-timestamp}(Q_k)$, the contents of Q_k are overwritten, otherwise a new version of Q is created.
- Reads always succeed; a write by T_i is rejected if some other transaction T_j that (in the serialization order defined by the timestamp values) should read T_i 's write, has already read a version created by a transaction older than T_i .

Multiversion Two-Phase Locking

- Differentiates between read-only transactions and update transactions
- *Update transactions* acquire read and write locks, and hold all locks up to the end of the transaction. That is, update transactions follow rigorous two-phase locking.
 - Each successful **write** results in the creation of a new version of the data item written.
 - each version of a data item has a single timestamp whose value is obtained from a counter **ts-counter** that is incremented during commit processing.
- *Read-only transactions* are assigned a timestamp by reading the current value of **ts-counter** before they start execution; they follow the multiversion timestamp-ordering protocol for performing reads.

Multiversion Two-Phase Locking

- When an update transaction wants to read a data item, it obtains a shared lock on it, and reads the latest version.
- When it wants to write an item, it obtains X lock on; it then creates a new version of the item and sets this version's timestamp to ∞ .
- When update transaction T_i completes, commit processing occurs:
 - T_i sets timestamp on the versions it has created to **ts-counter** + 1
 - T_i increments **ts-counter** by 1
- Read-only transactions that start after T_i increments **ts-counter** will see the values updated by T_i .
- Read-only transactions that start before T_i increments the **ts-counter** will see the value before the updates by T_i .
- Only serializable schedules are produced.

Multiversion Two-Phase Locking

- When an update transaction wants to read a data item, it obtains a shared lock on it, and reads the latest version.
- When it wants to write an item, it obtains X lock on; it then creates a new version of the item and sets this version's timestamp to ∞ .
- When update transaction T_i completes, commit processing occurs:
 - T_i sets timestamp on the versions it has created to **ts-counter** + 1
 - T_i increments **ts-counter** by 1
- Read-only transactions that start after T_i increments **ts-counter** will see the values updated by T_i .
- Read-only transactions that start before T_i increments the **ts-counter** will see the value before the updates by T_i .
- Only serializable schedules are produced.



Phantom Phenomenon

Beyond Syllabus



Insert/Delete Operations and Predicate Reads

- Locking rules for insert/delete operations
 - An exclusive lock must be obtained on an item before it is deleted
 - A transaction that inserts a new tuple into the database is automatically given an X-mode lock on the tuple
- Ensures that
 - reads/writes conflict with deletes
 - Inserted tuple is not accessible by other transactions until the transaction that inserts the tuple commits

Phantom Phenomenon

- Example of **phantom phenomenon**.
 - A transaction T1 that performs **predicate read** (or scan) of a relation
 - `select count(*)
from instructor
where dept_name = 'Physics'`
 - and a transaction T2 that inserts a tuple while T1 is active but after predicate read
 - `insert into instructor values ('11111', 'Feynman', 'Physics', 94000)`
(conceptually) conflict in spite of not accessing any tuple in common.
- If only tuple locks are used, non-serializable schedules can result
 - E.g. the scan transaction does not see the new instructor, but may read some other tuple written by the update transaction
- Can also occur with updates
 - E.g. update Wu's department from Finance to Physics

Insert/Delete Operations and Predicate Reads

- **Another Example:** T1 and T2 both find maximum instructor ID in parallel, and create new instructors with $ID = \text{maximum ID} + 1$
 - Both instructors get same ID, not possible in serializable schedule
- Schedule

T1	T2
Read(instructor where dept_name='Physics')	
	Insert Instructor in Physics
	Insert Instructor in Comp. Sci.
	Commit
Read(instructor where dept_name='Comp. Sci.')	

Handling Phantoms

- There is a conflict at the data level
 - The transaction performing predicate read or scanning the relation is reading information that indicates what tuples the relation contains
 - The transaction inserting/deleting/updating a tuple updates the same information.
 - The conflict should be detected, e.g. by locking the information.
- One solution:
 - Associate a data item with the relation, to represent the information about what tuples the relation contains.
 - Transactions scanning the relation acquire a shared lock in the data item,
 - Transactions inserting or deleting a tuple acquire an exclusive lock on the data item. (Note: locks on the data item do not conflict with locks on individual tuples.)
- Above protocol provides very low concurrency for insertions/deletions.

Index Locking To Prevent Phantoms

- **Index locking protocol** to prevent phantoms
 - Every relation must have at least one index.
 - A transaction can access tuples only after finding them through one or more indices on the relation
 - A transaction T_i that performs a lookup must lock all the index leaf nodes that it accesses, in S-mode
 - Even if the leaf node does not contain any tuple satisfying the index lookup (e.g. for a range query, no tuple in a leaf is in the range)
 - A transaction T_i that inserts, updates or deletes a tuple t_i in a relation r
 - Must update all indices to r
 - Must obtain exclusive locks on all index leaf nodes affected by the insert/update/delete
 - The rules of the two-phase locking protocol must be observed
- Guarantees that phantom phenomenon won't occur

Next-Key Locking to Prevent Phantoms

- Index-locking protocol to prevent phantoms locks entire leaf node
 - Can result in poor concurrency if there are many inserts
- **Next-key locking protocol:** provides higher concurrency
 - Lock all values that satisfy index lookup (match lookup value, or fall in lookup range)
 - Also lock next key value in index
 - even for inserts/deletes
 - Lock mode: S for lookups, X for insert/delete/update
- Ensures detection of query conflicts with inserts, deletes and updates

Consider B+-tree leaf nodes as below, with query predicate $7 \leq X \leq 16$.
Check what happens with next-key locking when inserting: (i) 15 and (ii) 7



Next-Key Locking to Prevent Phantoms

- Index-locking protocol to prevent phantoms locks entire leaf node
 - Can result in poor concurrency if there are many inserts
- **Next-key locking protocol:** provides higher concurrency
 - Lock all values that satisfy index lookup (match lookup value, or fall in lookup range)
 - Also lock next key value in index
 - even for inserts/deletes
 - Lock mode: S for lookups, X for insert/delete/update
- Ensures detection of query conflicts with inserts, deletes and updates

Consider B+-tree leaf nodes as below, with query predicate $7 \leq X \leq 16$.
Check what happens with next-key locking when inserting: (i) 15 and (ii) 7





Thank you

Concurrency Control
BCSE302L - Database
Systems.